

We Donate: Adama Community Support and Charity Management System

Adama City Administration

2026

Table of Contents

1	Introduction	5
1.1	Existing System	6
1.2	Limitations of Existing System	6
1.2.1	Lack of Transparency in Donation Management	7
1.2.2	Difficulty in Beneficiary Verification	7
1.2.3	Poor Donation Tracking and Monitoring	7
1.2.4	Manual Record Management	7
1.2.5	Limited Reporting and Data Analysis	7
1.2.6	Lack of Centralized Communication	7
1.2.7	Increased Risk of Fraud and Duplicate Support	8
1.2.8	High Administrative Workload	8
1.2.9	Limited Accessibility and Convenience	8
1.3	Objectives of Proposed System	8
1.3.1	General Objectives	8
1.3.2	Specific Objectives	9
1.4	Advantages of Proposed System	9
1.4.1	Improved Transparency	10
1.4.2	Efficient Beneficiary Verification	10
1.4.3	Centralized Data Management	10
1.4.4	Better Donation Tracking and Management	10
1.4.5	Automated Reporting and Analytics	10
1.4.6	Increased Accountability	10
1.4.7	Reduced Administrative Workload	10
1.4.8	Enhanced Security	11
1.4.9	Improved Communication and Coordination	11
1.4.10	Increased Donor Participation	11
1.4.11	Support for Smart City Transformation	11
1.4.12	Improved Decision Making	11
2	Methodology and Tools	12
2.1	Methodology	12
2.2	Tools	13
2.2.1	Frontend Technologies	13
2.2.2	Backend Technologies	13
2.2.3	Database Management System	13
2.2.4	Authentication and Security	14
2.2.5	Development Tools	14

2.2.6	Hosting and Deployment	14
3	System Requirement Specification	15
3.1	Overview	15
3.2	Feasibility Study	17
3.2.1	Technical Feasibility	17
3.2.2	Economic Feasibility	18
3.2.3	Operational Feasibility	18
3.2.4	Organizational Feasibility	19
3.2.5	Schedule Feasibility	19
3.3	Functional Requirements	20
3.3.1	User Authentication and Authorization	20
3.3.2	User Profile Management	21
3.3.3	Donor Management	21
3.3.4	Beneficiary Management	21
3.3.5	Support Request Management	22
3.3.6	Donation Management	23
3.3.7	Administration Management	23
3.3.8	Donation Tracking and Distribution Management	24
3.3.9	Notification Management	24
3.3.10	Reporting and Analytics	24
3.3.11	Audit and Activity Logging	25
3.3.12	File Management	25
3.3.13	Data Management	25
3.4	Non-Functional Requirements	26
3.4.1	Security Requirements	26
3.4.2	Performance Requirements	26
3.4.3	Usability Requirements	27
3.4.4	Reliability Requirements	27
3.4.5	Availability Requirements	27
3.4.6	Scalability Requirements	28
3.4.7	Maintainability Requirements	28
3.4.8	Compatibility Requirements	28
3.4.9	Data Integrity Requirements	28
3.4.10	Privacy Requirements	29
3.4.11	Portability Requirements	29
3.4.12	Auditability Requirements	29
4	System Architecture	30

4.1	System Overview Architecture	30
4.1.1	Presentation Layer (Frontend)	30
4.1.2	Business Logic Layer (Backend)	30
4.1.3	Data Storage Layer	31
4.2	System Workflow	31
4.2.1	Donor Workflow	31
4.2.2	Beneficiary Workflow	32
4.2.3	Administrator Workflow	32
5	Database Design	32
5.1	Entity Relationship Diagram	32
5.1.1	Users Table	32
5.1.2	Beneficiaries Table	32
5.1.3	Support Requests Table	33
5.1.4	Donations Table	33
5.1.5	Distributions Table	33
5.1.6	Notifications Table	33
5.1.7	Audit Logs Table	33
5.2	Database Relationships	33
6	Security Considerations	33
6.1	Authentication and Authorization	33
6.2	Data Security	34
6.3	Session Management	34
7	Conclusion	34

1 Introduction

Charitable giving and social support programs play a significant role in improving the quality of life of vulnerable individuals and promoting community development. Governments, non-governmental organizations (NGOs), companies, and individual donors frequently contribute financial resources, goods, and services to support people facing economic hardship, disabilities, natural disasters, health challenges, and other social problems. However, the effectiveness of these charitable activities largely depends on the availability of a transparent, accountable, and well-organized management system.

In many communities, traditional charity management is still carried out using manual or semi-digital methods. These approaches often lead to challenges such as poor record management, lack of transparency in donation distribution, delays in verifying beneficiaries, duplicate support, limited communication between donors and administrators, and difficulty generating accurate reports. As the number of donations and beneficiaries increases, managing charity operations efficiently becomes increasingly difficult.

To address these challenges, this project proposes **We Donate: Adama Community Support and Charity Management System**, a web-based platform designed to digitize and streamline the entire charity management process within Adama City Administration. The system provides a centralized platform that enables donors to contribute financial assistance, physical items, and community services while allowing government administrators to verify beneficiaries, review support requests, manage donation distribution, and monitor the overall donation process.

Unlike conventional public charity platforms, the proposed system emphasizes fairness and accountability by ensuring that beneficiaries are identified and verified through the appropriate government administrative structures before receiving assistance. Every donation transaction, approval process, and distribution activity is recorded within the system to improve transparency, strengthen public trust, and ensure that support reaches those who genuinely need it.

Furthermore, the system incorporates secure user authentication, role-based access control, donation tracking, request management, reporting, notifications, and dashboard analytics to improve operational efficiency and decision-making. By replacing manual procedures with a modern digital solution, the system supports the digital transformation initiatives of Adama City Administration while enhancing the effectiveness, reliability, and sustainability of community support programs.

1.1 Existing System

Currently, charity and community support activities in many government offices and local administrative institutions are managed using manual procedures. Information about beneficiaries, donations, and support requests is typically recorded on paper documents, notebooks, spreadsheets, or separate files maintained by different administrative offices. The absence of a centralized digital platform makes it difficult to effectively manage, monitor, and track charitable activities.

When individuals or families require assistance, their information is usually collected manually by local administrative bodies such as Kebele offices. Verification processes, beneficiary records, and support requests are handled through paper-based documentation and physical communication between different administrative levels. Similarly, donors who wish to contribute financial support, physical items, or services often rely on direct communication with administrative offices or community representatives.

The process of matching donations with beneficiaries is also performed manually, requiring significant time and effort from administrators. Tracking the status of donations, confirming deliveries, generating reports, and maintaining historical records are often challenging due to the lack of an integrated information management system. As a result, obtaining accurate and up-to-date information regarding donations and beneficiaries becomes difficult.

Furthermore, communication among donors, beneficiaries, and administrative offices is largely dependent on face-to-face interactions, telephone calls, and paper-based reports. These traditional methods increase administrative workload and limit the efficiency, transparency, and accountability of charity management activities.

Therefore, there is a need for a modern computerized system that can automate donation management processes, improve transparency, maintain accurate records, and facilitate efficient coordination among all stakeholders involved in community support and charity services.

1.2 Limitations of Existing System

The existing manual charity management system has several limitations that affect the efficiency, transparency, and reliability of community support activities. Since most processes are handled using paper-based records, spreadsheets, and physical communication, managing a large number of donors, beneficiaries, and donation activities becomes difficult. The major limitations of the existing system are described below.

1.2.1 Lack of Transparency in Donation Management

In the existing system, donors have limited ability to track how their contributions are used after donation. Information about donation collection, allocation, and distribution is usually maintained manually, making it difficult to provide clear evidence of where and when donations are delivered. This reduces donor confidence and creates challenges in maintaining accountability.

1.2.2 Difficulty in Beneficiary Verification

The current system relies heavily on manual verification processes conducted by local administrative offices. Collecting beneficiary information, checking eligibility, and maintaining updated records require significant time and effort. Due to the absence of a centralized database, identifying genuine beneficiaries and avoiding incorrect or duplicate registrations becomes challenging.

1.2.3 Poor Donation Tracking and Monitoring

The existing approach does not provide an effective mechanism to monitor the status of donations from the time they are submitted until they reach beneficiaries. Administrators may face difficulties tracking pending donations, completed distributions, and delivery confirmations, which can result in delays and inefficient resource allocation.

1.2.4 Manual Record Management

Most charity-related information is stored in physical documents, notebooks, or separate files. Managing and updating these records requires considerable administrative effort. Manual record keeping also increases the possibility of data loss, duplication, inconsistency, and human errors.

1.2.5 Limited Reporting and Data Analysis

Generating reports about donations, beneficiaries, distribution activities, and community needs is difficult in the existing system. Administrators need to manually collect and organize information from different sources, which consumes time and may result in inaccurate reports. The lack of analytical information also affects decision-making and planning.

1.2.6 Lack of Centralized Communication

Communication between donors, beneficiaries, NGOs, and administrative offices is mainly performed through direct meetings, phone calls, or informal channels. The absence of

a centralized communication platform makes it difficult to share updates, notifications, and important information efficiently.

1.2.7 Increased Risk of Fraud and Duplicate Support

Since beneficiary information and donation records are not managed through an integrated system, there is a risk of fraudulent requests, repeated assistance to the same beneficiaries, and improper allocation of resources. The lack of digital verification and audit mechanisms reduces control over charity activities.

1.2.8 High Administrative Workload

Manual processes require administrators to spend significant time collecting data, verifying beneficiaries, managing requests, and preparing reports. This increases operational costs and reduces the efficiency of government offices responsible for coordinating community support programs.

1.2.9 Limited Accessibility and Convenience

Donors and stakeholders cannot easily access updated information about available support requests, donation opportunities, or the progress of charity activities. The dependence on physical presence and manual communication limits participation and reduces the overall reach of charity services.

1.3 Objectives of Proposed System

The proposed **We Donate: Adama Community Support and Charity Management System** aims to overcome the limitations of the existing manual charity management approach by providing a centralized, transparent, and efficient digital platform for managing community support activities. The system focuses on improving donation management, beneficiary verification, communication, reporting, and accountability among donors, beneficiaries, and administrative offices.

1.3.1 General Objectives

The general objective of this project is to develop a web-based charity management system that digitizes and improves the process of donation collection, beneficiary verification, distribution management, and monitoring within Adama City Administration.

The proposed system aims to create a reliable platform that connects donors with verified beneficiaries through government administrative structures, ensuring transparency, fairness, and accountability in community support activities.

1.3.2 Specific Objectives

The specific objectives of the proposed system are:

- To develop an online platform for managing different types of donations, including financial donations, physical items, and community services.
- To provide a centralized database for storing and managing information about donors, beneficiaries, donations, requests, and distribution activities.
- To enable government administrative offices to register, verify, and approve beneficiaries before they receive community support.
- To improve transparency by allowing donation activities to be tracked from submission to final delivery.
- To reduce fraudulent requests and duplicate support through proper beneficiary verification and record management.
- To automate charity management processes that are currently performed manually, reducing administrative workload and improving efficiency.
- To provide role-based access control for different users, including donors, administrators, and other stakeholders, to ensure secure system usage.
- To generate accurate reports and analytical information to support decision-making and planning by administrators.
- To improve communication between donors, beneficiaries, and administrative offices through notifications and system updates.
- To maintain accountability by recording important activities, approvals, and transactions with responsible users and timestamps.
- To support the digital transformation initiatives of Adama City Administration by introducing a modern and sustainable charity management solution.

1.4 Advantages of Proposed System

The proposed **We Donate: Adama Community Support and Charity Management System** provides several advantages over the existing manual charity management approach. By introducing a centralized digital platform, the system improves the efficiency, transparency, security, and reliability of community support activities. The major advantages of the proposed system are described below.

1.4.1 Improved Transparency

The system provides clear visibility of donation activities by allowing donations to be tracked from the initial submission until final delivery to beneficiaries. Donors and administrators can monitor donation status, distribution progress, and completion records, which increases trust and confidence in charity operations.

1.4.2 Efficient Beneficiary Verification

The proposed system enables authorized government administrators to register and verify beneficiaries before they receive support. This helps ensure that assistance reaches genuine individuals and families while reducing fraudulent requests and duplicate support.

1.4.3 Centralized Data Management

All information related to donors, beneficiaries, donations, requests, and distribution activities is stored in a centralized database. This reduces dependence on paper records, minimizes data duplication, and makes information retrieval faster and more reliable.

1.4.4 Better Donation Tracking and Management

The system allows administrators to monitor the complete lifecycle of donations, including donation creation, approval, assignment, delivery, and completion. This improves resource management and ensures that donations are properly allocated.

1.4.5 Automated Reporting and Analytics

The proposed system can automatically generate reports and provide analytical information about donation activities, beneficiary needs, and distribution performance. This supports better decision-making and helps administrators plan effective community support programs.

1.4.6 Increased Accountability

Every important activity performed within the system is recorded with user information and timestamps. This creates an audit trail that helps administrators monitor activities, identify problems, and maintain responsibility throughout the donation process.

1.4.7 Reduced Administrative Workload

By automating manual tasks such as record keeping, request management, verification tracking, and report generation, the system reduces the workload of administrative staff and improves operational efficiency.

1.4.8 Enhanced Security

The system provides secure authentication and role-based access control to ensure that users can only access features and information appropriate to their responsibilities. This protects sensitive beneficiary and donation information from unauthorized access.

1.4.9 Improved Communication and Coordination

The platform improves interaction between donors, beneficiaries, NGOs, and administrative offices through centralized information sharing and notification mechanisms. This reduces delays caused by traditional communication methods.

1.4.10 Increased Donor Participation

By providing an accessible online platform, donors can easily view available support needs, contribute donations, track their contributions, and understand the impact of their support. This encourages greater community participation.

1.4.11 Support for Smart City Transformation

The proposed system contributes to the digital transformation goals of Adama City Administration by replacing manual charity management processes with a modern, data-driven, and sustainable digital solution.

1.4.12 Improved Decision Making

Through accurate records, reports, and analytics, administrators can better understand community needs, identify priority areas, and make informed decisions regarding resource allocation and social support programs.

2 Methodology and Tools

2.1 Methodology

The development of the **We Donate: Adama Community Support and Charity Management System** follows an **Agile Software Development Methodology**. Agile is an iterative and incremental approach that allows the system to be developed, tested, and improved through multiple development cycles.

The Agile methodology is selected because the proposed system consists of different modules, including user authentication, user management, donation management, beneficiary verification, request management, reporting, and dashboard analytics. Developing the system in small iterations allows the development team to implement, test, and refine each module based on requirements and feedback.

The major activities followed during the development process include:

Requirement Analysis During this phase, the requirements of the system are collected and analyzed by identifying the problems of the existing charity management system and defining the needs of different stakeholders, including donors, beneficiaries, and administrative users.

System Design In this phase, the overall architecture, database structure, user interfaces, and system workflows are designed. The design focuses on creating a secure and user-friendly platform that supports efficient charity management.

Implementation The system is developed by implementing different modules incrementally. Each module is developed, integrated, and tested before moving to the next development stage. The main implemented modules include authentication, donation management, beneficiary management, administration, reporting, and dashboards.

Testing Testing is performed throughout the development process to ensure that each system component functions correctly. Different testing approaches, including unit testing, integration testing, system testing, and user acceptance testing, are considered to verify system quality and reliability.

Deployment and Maintenance After successful testing, the system is prepared for deployment. Continuous maintenance and improvements are performed based on user feedback and future requirements.

2.2 Tools

The following tools and technologies are used for the development of the **We Donate: Adama Community Support and Charity Management System**.

2.2.1 Frontend Technologies

React.js React.js is used for developing the user interface of the system. It provides reusable components and enables the development of interactive dashboards and responsive web pages for different users.

HTML5 and CSS3 HTML5 and CSS3 are used for structuring and designing the web interfaces.

JavaScript / TypeScript JavaScript and TypeScript are used to implement client-side functionality and improve code reliability through type checking.

Tailwind CSS Tailwind CSS is used for creating responsive and modern user interface designs efficiently.

2.2.2 Backend Technologies

Node.js Node.js is used as the backend runtime environment for building server-side applications and handling API requests.

Express.js Express.js is used as a backend framework to develop RESTful APIs and manage communication between the frontend application and database.

2.2.3 Database Management System

PostgreSQL PostgreSQL is used as the primary database management system for storing and managing system data. It provides reliable data storage, strong security features, scalability, and support for complex relational data structures. The database stores information related to users, organizations, beneficiaries, donations, requests, transactions, notifications, and system activities.

Prisma ORM Prisma ORM is used to interact with the PostgreSQL database. It simplifies database operations, provides type-safe queries, and helps manage database schemas and migrations efficiently.

2.2.4 Authentication and Security

JSON Web Token (JWT) JWT is used for secure user authentication and authorization. It enables role-based access control for different users, such as donors, administrators, NGOs, and members.

bcrypt bcrypt is used for securely hashing user passwords before storing them in the database.

2.2.5 Development Tools

- **Visual Studio Code** - Used as the primary code editor for system development.
- **Git and GitHub** - Used for source code management, version control, and team collaboration.
- **Postman** - Used for testing and validating backend APIs.
- **Figma** - Used for designing and prototyping the system user interfaces.
- **Vite** - Used as the frontend development and build tool for creating a fast and efficient React application.

2.2.6 Hosting and Deployment

Cloud Server The system can be deployed on a cloud server to make the application accessible online and support scalability.

3 System Requirement Specification

A **System Requirement Specification (SRS)** is a document that describes the complete requirements of a software system, including its expected functionalities, constraints, and quality attributes. It serves as a communication document between stakeholders, developers, and users by defining what the system should accomplish and how it should behave.

This chapter presents the system requirements for the **We Donate: Adama Community Support and Charity Management System**. The requirements are identified based on the analysis of the existing charity management process and the objectives of the proposed system.

The purpose of this chapter is to define the major requirements needed for designing and developing the proposed system. It includes an overview of the system, feasibility study, functional requirements, and non-functional requirements.

The We Donate system is a web-based charity management platform designed to improve the transparency, efficiency, and accountability of community support activities within Adama City Administration. The system provides different functionalities for donors, beneficiaries, administrative offices, NGOs, and system administrators.

The system requirements focus on automating donation management, beneficiary verification, request processing, distribution tracking, reporting, and secure information management. By implementing these requirements, the system aims to provide a reliable digital platform that ensures donations are properly managed and delivered to verified beneficiaries.

3.1 Overview

The **We Donate: Adama Community Support and Charity Management System** is a web-based platform developed to improve the management of charity activities and community support services within Adama City Administration. The system provides a centralized solution for managing donors, beneficiaries, donations, requests, verification processes, and distribution activities.

The main purpose of the system is to replace traditional manual charity management processes with a digital platform that improves transparency, efficiency, and accountability. The system enables donors to contribute financial donations, physical items, and services while ensuring that support is provided to verified beneficiaries through authorized administrative structures.

The system consists of different modules that support the activities of various stakehold-

ers. Each user type has specific roles and permissions based on their responsibilities.

The major system users include:

Donor Donors are individuals, companies, NGOs, diaspora members, or organizations that provide support to the community. The system allows donors to create accounts, manage profiles, view available support requests, make donations, track donation progress, and access donation history and receipts.

Beneficiary Beneficiaries are individuals or groups who require community support, such as vulnerable families, disabled individuals, elderly people, students, or disaster-affected people. Beneficiary information is verified by authorized administrators before receiving support. The system allows beneficiaries to submit requests, upload supporting documents, and monitor request status.

Administrative Users Administrative users are responsible for managing and supervising charity activities. The system supports different administrative levels, including:

- **Kebele Administrators:** Responsible for registering beneficiaries, collecting information, verifying households, and confirming local delivery activities.
- **Woreda Administrators:** Responsible for monitoring kebele activities, reviewing requests, approving distributions, and managing reports.
- **City Administrators:** Responsible for overall system management, user management, donation monitoring, reporting, auditing activities, and configuring system settings.

NGO / Organization Users NGOs and organizations can participate in community support activities by providing donations, services, or resources. They can manage their organizational information and participate in approved charity programs.

System Administrator The system administrator manages the technical operation of the platform, including user accounts, system security, access control, and configuration management.

The major functions provided by the system include:

- Secure user registration and authentication
- Role-based access control
- Beneficiary registration and verification
- Support request management

- Donation creation and tracking
- Donation distribution monitoring
- Notification and communication management
- Report generation and analytics
- Audit trail and activity monitoring
- Data management and security

The system is designed using a modern web application architecture with React.js for the frontend, Node.js and Express.js for the backend, and PostgreSQL as the database management system. The system uses secure authentication mechanisms and structured data management approaches to ensure reliable and efficient charity operations.

Overall, the proposed system provides a digital foundation for improving community support services in Adama City Administration by ensuring that donations are managed transparently and delivered effectively to verified beneficiaries.

3.2 Feasibility Study

A feasibility study is conducted to evaluate whether the proposed **We Donate: Adama Community Support and Charity Management System** is practical, achievable, and suitable for implementation. It analyzes the project from different perspectives, including technical, economic, operational, and organizational feasibility.

The purpose of the feasibility study is to identify whether the available resources, technologies, and organizational conditions are sufficient to successfully develop and operate the proposed system.

3.2.1 Technical Feasibility

The proposed system is technically feasible because the required technologies, development tools, and infrastructure are available for implementation. The system can be developed using modern web technologies that support scalability, security, and maintainability.

The technologies used for developing the system include:

- **Frontend:** React.js, TypeScript, HTML5, CSS3, and Tailwind CSS for developing responsive and interactive user interfaces.
- **Backend:** Node.js and Express.js for developing server-side logic and RESTful APIs.

- **Database:** PostgreSQL for storing and managing system information, including users, beneficiaries, donations, requests, and transaction records.
- **Database Management:** Prisma ORM for efficient and secure interaction between the application and PostgreSQL database.
- **Authentication:** JSON Web Token (JWT) and bcrypt for secure authentication and password protection.

The development team has the necessary programming knowledge and access to required software tools such as Visual Studio Code, GitHub, Postman, and cloud development environments. Therefore, the project is technically feasible.

3.2.2 Economic Feasibility

The proposed system is economically feasible because it reduces the costs associated with traditional manual charity management processes. The current manual approach requires resources for paper documentation, physical storage, manual reporting, and administrative activities.

The proposed digital system reduces these costs by automating record management, report generation, donation tracking, and communication processes.

Most technologies used in the development process are open-source and freely available, including:

- React.js
- Node.js
- Express.js
- PostgreSQL
- Prisma ORM
- Visual Studio Code
- Git and GitHub

This reduces software licensing costs and makes the system affordable to develop and maintain.

Therefore, the proposed system is economically feasible.

3.2.3 Operational Feasibility

The proposed system is operationally feasible because it addresses the major challenges found in the existing manual charity management process. The system provides an easy-

to-use platform that improves coordination between donors, beneficiaries, NGOs, and administrative offices.

The system simplifies charity management activities by allowing users to:

- Register and manage accounts
- Submit and review support requests
- Verify beneficiaries
- Manage donations
- Track donation distribution
- Generate reports
- Monitor charity activities

Role-based dashboards allow each user to access only the functions related to their responsibilities, making the system easier to use and manage.

Therefore, the system is operationally feasible and can improve the efficiency of charity operations.

3.2.4 Organizational Feasibility

The proposed system is organizationally feasible because it supports the goals and responsibilities of Adama City Administration in managing community support programs.

The system improves coordination among different administrative levels, including Kebele, Woreda, and City Administration offices, by providing a centralized platform for managing beneficiary information, donations, and reports.

The system also supports digital transformation initiatives by replacing manual procedures with a modern computerized solution that improves transparency, accountability, and decision-making.

Therefore, the proposed system is suitable for adoption within the organizational structure of Adama City Administration.

3.2.5 Schedule Feasibility

The proposed system is schedule feasible because the project can be developed within the available project timeline using an incremental development approach. By dividing the system into smaller modules, the development team can implement, test, and integrate each component systematically.

The major development phases include:

- Requirement analysis
- System design
- Database design
- Frontend and backend development
- Module integration
- Testing and deployment

Using Agile methodology allows continuous progress monitoring and adjustment throughout the development process.

3.3 Functional Requirements

Functional requirements describe the specific services and operations that the **We Donate: Adama Community Support and Charity Management System** must provide to its users. These requirements define what the system should do to support charity management activities, including user management, donation processing, beneficiary verification, distribution tracking, reporting, and system administration.

The functional requirements are identified based on the needs of different system users, including donors, beneficiaries, administrative users, NGOs/organizations, and system administrators.

3.3.1 User Authentication and Authorization

The system shall provide secure authentication and authorization mechanisms to ensure that only registered users can access system features according to their roles.

The system shall:

- Allow users to register an account based on their user type.
- Allow users to log in using valid credentials.
- Allow users to log out securely.
- Provide password encryption for user security.
- Manage user sessions securely.
- Provide role-based access control for different users.
- Restrict unauthorized users from accessing protected system resources.

3.3.2 User Profile Management

The system shall allow users to manage their personal and organizational information.

The system shall:

- Allow users to view their profile information.
- Allow users to update profile details.
- Allow organizations and NGOs to manage organizational information.
- Maintain user account status and information records.

3.3.3 Donor Management

The system shall provide functionalities for donors to participate in community support activities.

The system shall allow donors to:

- Register and manage donor accounts.
- View available donation opportunities.
- Browse verified support requests.
- Create donation records.
- Donate financial resources.
- Donate physical items.
- Offer volunteer services or professional skills.
- View donation history.
- Track the progress and status of donations.
- Receive donation confirmations and digital receipts.

3.3.4 Beneficiary Management

The system shall support the registration, verification, and management of beneficiary information.

The system shall allow authorized administrators to:

- Register beneficiary information.
- Create beneficiary profiles.
- Verify beneficiary eligibility.

- Upload and manage supporting documents.
- Update beneficiary records.
- View beneficiary history.
- Prevent duplicate beneficiary registration.

The system shall allow beneficiaries to:

- Submit support requests.
- View request status.
- Provide required information and documents.
- Receive notifications regarding request updates.

3.3.5 Support Request Management

The system shall provide a mechanism for managing community support requests.

The system shall:

- Allow beneficiaries or authorized administrators to create support requests.
- Allow administrators to review submitted requests.
- Allow administrators to approve or reject requests.
- Maintain request status throughout the process.

The request status shall include:

- Draft
- Submitted
- Under Review
- Approved
- Published
- Partially Funded
- Fully Funded
- Completed
- Rejected

3.3.6 Donation Management

The system shall manage the complete donation lifecycle from creation to completion.

The system shall:

- Allow donors to create donation records.
- Categorize donations by type:
 - Money donation
 - Physical item donation
 - Service donation
- Record donation details.
- Assign donations to approved support requests.
- Track donation status.
- Confirm successful delivery.
- Maintain donation history.

3.3.7 Administration Management

The system shall provide administrative functionalities for managing charity activities.

Kebele Administrator Functions The system shall allow Kebele administrators to:

- Register beneficiaries.
- Verify household information.
- Review local support requests.
- Recommend eligible beneficiaries.
- Confirm donation delivery.

Woreda Administrator Functions The system shall allow Woreda administrators to:

- Monitor Kebele-level activities.
- Review and approve distributions.
- Manage reports.
- Handle support-related issues.

City Administrator Functions The system shall allow City administrators to:

- Manage users and roles.
- Monitor all donation activities.
- View city-level statistics.
- Generate reports.
- Audit system activities.
- Manage system settings.

3.3.8 Donation Tracking and Distribution Management

The system shall provide tracking mechanisms to monitor donations from submission to final delivery.

The system shall:

- Record donation progress.
- Track assigned beneficiaries.
- Monitor distribution status.
- Record delivery confirmation.
- Maintain distribution history.

3.3.9 Notification Management

The system shall provide notifications to keep users informed about important activities.

The system shall:

- Notify users about account activities.
- Notify donors about donation status changes.
- Notify administrators about pending requests.
- Notify beneficiaries about request approval or rejection.

3.3.10 Reporting and Analytics

The system shall provide reporting and analytical features to support decision-making.

The system shall:

- Generate donation reports.

- Generate beneficiary reports.
- Provide donation statistics.
- Display dashboard analytics.
- Support monitoring of charity activities.
- Provide information for planning and decision-making.

3.3.11 Audit and Activity Logging

The system shall maintain records of important user activities.

The system shall:

- Record user actions.
- Store activity timestamps.
- Track approval and verification activities.
- Maintain audit logs for accountability and transparency.

3.3.12 File Management

The system shall support uploading and managing required documents.

The system shall:

- Allow users to upload supporting documents.
- Store beneficiary verification documents.
- Manage uploaded files securely.
- Allow authorized users to access required documents.

3.3.13 Data Management

The system shall provide reliable data storage and management.

The system shall:

- Store user information securely.
- Maintain donation and beneficiary records.
- Support data retrieval and updating.
- Prevent data inconsistency and duplication.

These functional requirements define the core services that the We Donate system must provide to achieve its goal of creating a transparent, accountable, and efficient charity management platform.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes, performance expectations, security standards, and operational characteristics of the **We Donate: Adama Community Support and Charity Management System**. Unlike functional requirements that describe what the system should do, non-functional requirements describe how the system should perform and the standards it should satisfy.

The major non-functional requirements of the proposed system are described below.

3.4.1 Security Requirements

The system shall provide security mechanisms to protect user information, donation records, and other sensitive data.

The system shall:

- Provide secure user authentication using JWT-based authentication.
- Encrypt user passwords before storing them in the database using secure hashing techniques.
- Implement role-based access control to restrict users based on their responsibilities.
- Prevent unauthorized access to system resources.
- Protect beneficiary and donor information from unauthorized disclosure.
- Maintain secure communication between system components.
- Record important user activities through audit logs for accountability.

3.4.2 Performance Requirements

The system shall provide efficient performance to support users and administrative operations.

The system shall:

- Provide fast response time for user requests and system operations.
- Efficiently process donation records, beneficiary information, and reports.
- Support multiple users accessing the system simultaneously.

- Optimize database operations to ensure quick data retrieval and storage.
- Maintain acceptable performance as the amount of system data increases.

3.4.3 Usability Requirements

The system shall provide an easy-to-use and user-friendly interface for different categories of users.

The system shall:

- Provide simple and clear navigation.
- Provide responsive interfaces that work on different screen sizes.
- Use understandable labels and instructions.
- Provide dashboards based on user roles.
- Reduce the complexity of charity management tasks.
- Allow users with limited technical experience to use the system effectively.

3.4.4 Reliability Requirements

The system shall provide reliable services and maintain data accuracy during operation.

The system shall:

- Ensure accurate storage and retrieval of information.
- Prevent data loss during normal operations.
- Maintain consistency of donation and beneficiary records.
- Handle errors properly and provide meaningful error messages.
- Perform regular data backup and recovery when required.

3.4.5 Availability Requirements

The system shall be available whenever users need to access charity management services.

The system shall:

- Provide continuous access to authorized users.
- Minimize system downtime.
- Support reliable deployment on cloud infrastructure.
- Ensure that users can access donation and request information when required.

3.4.6 Scalability Requirements

The system shall be designed to support future growth in users, donations, and system data.

The system shall:

- Support increasing numbers of donors, beneficiaries, and administrators.
- Allow additional features and modules to be integrated in the future.
- Handle increased database size without significant performance degradation.
- Support expansion to other administrative areas if required.

3.4.7 Maintainability Requirements

The system shall be easy to maintain, modify, and improve after deployment.

The system shall:

- Use modular system architecture.
- Follow clean coding practices and documentation standards.
- Allow developers to easily identify and fix errors.
- Support future updates and feature improvements.
- Use organized database structures and reusable components.

3.4.8 Compatibility Requirements

The system shall be compatible with commonly used technologies and platforms.

The system shall:

- Support modern web browsers such as Chrome, Firefox, Edge, and Safari.
- Work across different operating systems.
- Provide responsive access through computers, tablets, and mobile devices.
- Support integration with future external services if required.

3.4.9 Data Integrity Requirements

The system shall maintain the accuracy and consistency of stored information.

The system shall:

- Validate user input before storing data.

- Prevent duplicate records.
- Maintain relationships between users, donations, beneficiaries, and requests.
- Ensure that completed transactions cannot be modified without authorization.

3.4.10 Privacy Requirements

The system shall protect the privacy of users and beneficiaries.

The system shall:

- Collect only necessary user information.
- Restrict access to confidential beneficiary data.
- Ensure personal information is visible only to authorized users.
- Follow responsible data management practices.

3.4.11 Portability Requirements

The system shall be portable and deployable in different environments.

The system shall:

- Support deployment on different cloud hosting platforms.
- Use widely supported technologies such as React.js, Node.js, and PostgreSQL.
- Allow easy migration and configuration in new environments.

3.4.12 Auditability Requirements

The system shall maintain traceability of important operations.

The system shall:

- Record user activities.
- Store timestamps for important actions.
- Track donation approval and distribution processes.
- Provide administrators with audit information when required.

The non-functional requirements ensure that the We Donate system is secure, reliable, scalable, and easy to use while maintaining high-quality service delivery for charity management activities.

4 System Architecture

4.1 System Overview Architecture

The **We Donate: Adama Community Support and Charity Management System** follows a modern three-tier architecture that separates the presentation layer, business logic layer, and data storage layer. This architectural approach ensures scalability, maintainability, and security of the system.

4.1.1 Presentation Layer (Frontend)

The presentation layer is built using React.js, a JavaScript library for building user interfaces. This layer is responsible for rendering the user interface, handling user interactions, and communicating with the backend services. The frontend application is designed to be responsive and accessible across various devices including desktop computers, tablets, and smartphones.

Key components of the presentation layer include:

- User authentication and registration interfaces
- Role-based dashboards (Donor, Beneficiary, Administrator, NGO)
- Donation management interfaces
- Request management interfaces
- Reporting and analytics dashboards
- Notification management interfaces

4.1.2 Business Logic Layer (Backend)

The business logic layer is implemented using Node.js with Express.js framework. This layer handles all server-side operations, business rules, and data processing. It provides RESTful APIs that the frontend application consumes.

Key functions of the business logic layer include:

- User authentication and authorization
- Donation processing and management
- Beneficiary verification and management
- Request handling and approval workflows
- Distribution tracking and management

- Report generation
- Notification management
- System administration functions

4.1.3 Data Storage Layer

The data storage layer uses PostgreSQL as the primary database management system. Prisma ORM is used to interact with the database, providing type-safe database operations and schema management.

The database stores the following major entities:

- Users (Donors, Beneficiaries, Administrators, NGOs)
- Beneficiary profiles
- Donation records
- Support requests
- Distribution records
- Notifications
- Audit logs
- System configurations

4.2 System Workflow

4.2.1 Donor Workflow

1. **Registration:** Donor registers by providing personal or organizational information.
2. **Profile Management:** Donor manages their profile and contact information.
3. **Browse Requests:** Donor views available support requests from verified beneficiaries.
4. **Make Donation:** Donor selects a request and makes a donation (financial, physical items, or services).
5. **Track Donation:** Donor monitors the status of their donation from submission to delivery.
6. **View History:** Donor accesses donation history and digital receipts.

4.2.2 Beneficiary Workflow

1. **Registration:** Beneficiary is registered by authorized administrator at Kebele level.
2. **Verification:** Beneficiary information is verified and confirmed.
3. **Submit Request:** Beneficiary submits a support request with supporting documents.
4. **Review:** Request is reviewed by administrators.
5. **Approval:** Request is approved or rejected based on eligibility.
6. **Receive Support:** Approved request is published for donors to support.
7. **Track Status:** Beneficiary monitors the status of their request.

4.2.3 Administrator Workflow

1. **User Management:** Administrators manage user accounts and roles.
2. **Beneficiary Management:** Register, verify, and manage beneficiary information.
3. **Request Management:** Review, approve, and manage support requests.
4. **Donation Management:** Monitor donations and ensure proper distribution.
5. **Reporting:** Generate reports and analytics for decision-making.
6. **Audit:** Monitor system activities and maintain accountability.

5 Database Design

5.1 Entity Relationship Diagram

The database for the **We Donate** system consists of the following major entities:

5.1.1 Users Table

Stores information about all system users including donors, beneficiaries, administrators, and NGOs.

5.1.2 Beneficiaries Table

Stores detailed information about registered beneficiaries including personal details, verification status, and supporting documents.

5.1.3 Support Requests Table

Stores information about support requests submitted by beneficiaries including request type, description, amount, status, and verification information.

5.1.4 Donations Table

Stores information about donations made by donors including donation type, amount/items, associated request, and delivery status.

5.1.5 Distributions Table

Stores information about donation distributions including allocation details, delivery confirmation, and verification.

5.1.6 Notifications Table

Stores notification records sent to users regarding system activities and updates.

5.1.7 Audit Logs Table

Stores activity logs for security, accountability, and transparency purposes.

5.2 Database Relationships

The key relationships between database entities include:

- One User can be associated with multiple Donations
- One Beneficiary can have multiple Support Requests
- One Support Request can receive multiple Donations
- One Donation can be assigned to one Distribution
- One User can receive multiple Notifications
- One User can have multiple Audit Logs

6 Security Considerations

6.1 Authentication and Authorization

The system implements secure authentication using JSON Web Tokens (JWT). Users must provide valid credentials (email and password) to access the system. Passwords are securely hashed using bcrypt before storage.

Role-based access control (RBAC) ensures that users can only access features and data appropriate to their role (Donor, Beneficiary, Administrator, NGO, or System Administrator).

6.2 Data Security

All sensitive data is encrypted both in transit (using HTTPS) and at rest. User passwords are never stored in plain text. Personal information of beneficiaries and donors is protected from unauthorized access.

6.3 Session Management

User sessions are managed securely with appropriate timeout periods. Users are automatically logged out after periods of inactivity to prevent unauthorized access.

7 Conclusion

The **We Donate: Adama Community Support and Charity Management System** provides a comprehensive digital solution for managing charity activities within Adama City Administration. By replacing manual processes with a modern web-based platform, the system improves transparency, accountability, efficiency, and coordination among all stakeholders involved in community support services.

The system addresses the major limitations of existing manual charity management approaches including lack of transparency, difficulty in beneficiary verification, poor donation tracking, manual record management, limited reporting capabilities, and high administrative workload.

The implementation of this system contributes to the digital transformation goals of Adama City Administration while enhancing the effectiveness and sustainability of community support programs. The system ensures that donations reach verified beneficiaries efficiently, maintains accurate records for accountability, and provides analytical information to support decision-making and planning.

Through its secure, scalable, and user-friendly design, the We Donate system creates a reliable platform that encourages greater community participation in charitable activities and strengthens public trust in government-administered support programs.